

Lab 6: Data Visualization and Interpretation using Plotly

```
In [1]: %pip install plotly
        %pip install -U nbformat ipykernel
```

```
Requirement already satisfied: plotly in
c:\users\pkg15\appdata\local\programs\python\python313\lib\site-packages (6.5.2)
Requirement already satisfied: narwhals>=1.15.1 in
c:\users\pkg15\appdata\local\programs\python\python313\lib\site-packages (from
plotly) (2.17.0)
Requirement already satisfied: packaging in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from plotly)
(25.0) Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Requirement already satisfied: nbformat in
 c:\users\pkg15\appdata\local\programs\python\python313\lib\site-packages
 (5.10.4)

Requirement already satisfied: ipykernel in
 c:\users\pkg15\appdata\local\programs\python\python313\lib\site-packages (7.2.0)

Requirement already satisfied: fastjsonschema>=2.15 in
 c:\users\pkg15\appdata\local
 \programs\python\python313\lib\site-packages (from nbformat) (2.21.2)

Requirement already satisfied: jsonschema>=2.6 in
 c:\users\pkg15\appdata\local\programs\python\python313\lib\site-packages (from
 nbformat) (4.26.0)

Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in
 c:\users\pkg15\appdata
 \roaming\python\python313\site-packages (from nbformat) (5.9.1)

Requirement already satisfied: traitlets>=5.1 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from nbformat)
 (5.14.3)

Requirement already satisfied: comm>=0.1.1 in
 c:\users\pkg15\appdata\roaming\python
 \python313\site-packages (from ipykernel) (0.2.3)

Requirement already satisfied: debugpy>=1.6.5 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from ipykernel)
 (1.8.19)

Requirement already satisfied: ipython>=7.23.1 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from ipykernel)
 (9.9.0)

Requirement already satisfied: jupyter-client>=8.8.0 in
 c:\users\pkg15\appdata\local
 \programs\python\python313\lib\site-packages (from ipykernel) (8.8.0)

Requirement already satisfied: matplotlib-inline>=0.1 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from ipykernel)
 (0.2.1)

Requirement already satisfied: nest-asyncio>=1.4 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from ipykernel)
 (1.6.0)

Requirement already satisfied: packaging>=22 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from ipykernel)
 (25.0)

Requirement already satisfied: psutil>=5.7 in
 c:\users\pkg15\appdata\roaming\python
 \python313\site-packages (from ipykernel) (7.2.1)

Requirement already satisfied: pyzmq>=25 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from ipykernel)
 (27.1.0)

Requirement already satisfied: tornado>=6.4.1 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from ipykernel)
 (6.5.4)

Requirement already satisfied: colorama>=0.4.4 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from
 ipython>=7.23.1->ipykernel) (0.4.6)

Requirement already satisfied: decorator>=4.3.2 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from
 ipython>=7.23.1->ipykernel) (5.2.1)

Requirement already satisfied: ipython-pygments-lexers>=1.0.0 in
 c:\users\pkg15\appdata\roaming\python\python313\site-packages (from

```

ipython>=7.23.1->ipykernel) (1.1.1) Requirement already satisfied: jedi>=0.18.1
in c:\users\pkg15\appdata\roaming\python
\python313\site-packages (from ipython>=7.23.1->ipykernel) (0.19.2)
Requirement already satisfied: prompt_toolkit<3.1.0,>=3.0.41 in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from
ipython>=7.23.1->ipykernel) (3.0.52) Requirement already satisfied:
pygments>=2.11.0 in c:\users\pkg15\appdata\roaming\python\python313\site-
packages (from ipython>=7.23.1->ipykernel) (2.19.2)
Requirement already satisfied: stack_data>=0.6.0 in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from
ipython>=7.23.1->ipykernel) (0.6.3)
Requirement already satisfied: wcwidth in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from
prompt_toolkit<3.1.0,>=3.0.41->ipython>=7.23.1->ipykernel) (0.2.14)
Requirement already satisfied: parso<0.9.0,>=0.8.4 in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from
jedi>=0.18.1->ipython>=7.23.1->ipykernel) (0.
8.5) Requirement already satisfied: attrs>=22.2.0 in
c:\users\pkg15\appdata\local\programs\python\python313\lib\site-packages (from
jsonschema>=2.6->nbformat) (25.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in
c:\users\pkg1
5\appdata\local\programs\python\python313\lib\site-packages (from jsonschema>=2.6-
>nbformat) (2025.9.1)
Requirement already satisfied: referencing>=0.28.4 in
c:\users\pkg15\appdata\local\programs\python\python313\lib\site-packages (from
jsonschema>=2.6->nbformat) (0.37.0) Requirement already satisfied: rpds-
py>=0.25.0 in c:\users\pkg15\appdata\local\programs\python\python313\lib\site-
packages (from jsonschema>=2.6->nbformat) (0.30.0)
Requirement already satisfied: python-dateutil>=2.8.2 in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from jupyter-
client>=8.8.0->ipykernel) (2.9.0.post0)
Requirement already satisfied: platformdirs>=2.5 in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from jupyter-
core!=5.0.*,>=4.12->nbformat) (4.5.1)
Requirement already satisfied: six>=1.5 in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from python-
dateutil>=2.8.2->jupyter-client>=8.8.0->ipykernel) (1.17.0)
Requirement already satisfied: executing>=1.2.0 in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from
stack_data>=0.6.0->ipython>=7.23.1->ipykernel) (2.2.1)
Requirement already satisfied: asttokens>=2.1.0 in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from
stack_data>=0.6.0->ipython>=7.23.1->ipykernel) (3.0.1)
Requirement already satisfied: pure-eval in
c:\users\pkg15\appdata\roaming\python\python313\site-packages (from
stack_data>=0.6.0->ipython>=7.23.1->ipykernel) (0.2.3) Note: you may need to
restart the kernel to use updated packages.

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

```

Importing the libraries

```
In [2]: import seaborn as sns
import plotly.express as px
import plotly.graph_objects as go
import pandas as pd
import matplotlib.pyplot as plt
```

Loading the dataset:- GapMinder Dataset

```
In [3]: df = px.data.gapminder()
df.head()
```

```
Out[3]:
```

	country	continent	year	lifeExp	pop	gdpPercap	iso_alpha	iso_num
0	Afghanistan	Asia	1952	28.801	8425333	779.445314	AFG	4
1	Afghanistan	Asia	1957	30.332	9240934	820.853030	AFG	4
2	Afghanistan	Asia	1962	31.997	10267083	853.100710	AFG	4
3	Afghanistan	Asia	1967	34.020	11537966	836.197138	AFG	4
4	Afghanistan	Asia	1972	36.088	13079460	739.981106	AFG	4

Tasks

1. Scatter Plot with Slider (Animation)

- Plot GDP per capita (gdpPercap) vs life expectancy (lifeExp) for all countries.
- Use population (pop) to size the points.
- Color points by continent.
- Add a slider to move through different years.
- Title: "GDP vs Life Expectancy Over Time".
- Label the x-axis as "GDP per Capita (USD)" and the y-axis as "Life Expectancy (Years)".
- Ensure hover tooltips display country names.

```
In [4]: px.scatter(df, x="gdpPercap",
y="lifeExp", animation_frame="year",
animation_group="country", size="pop",
color="continent",
hover_name="country", title="GDP vs
Life Expectancy Over Time", labels={
    "gdpPercap": "GDP per Capita (USD)",
    "lifeExp": "Life Expectancy (Years)",
    "continent": "Continent",
    "pop": "Population",
    "year": "Year",
},
size_max=60,
)
```

2. Line Plot with Dropdown Selector

- Plot life expectancy (lifeExp) vs year (year) for multiple countries (e.g., India, China, USA, Brazil).
- Add a dropdown menu to select which country's line is visible.
- Use distinct line colors for each country.
- Title: "Life Expectancy Comparison by Country".
- Label the x-axis as "Year" and the y-axis as "Life Expectancy (Years)".
- Place the legend at the bottom

```
In [1]: import plotly.express as px

df = px.data.gapminder()

countries = ["India", "China", "United States", "Brazil"]
df_selected = df[df["country"].isin(countries)]

fig = px.line(
df_selected,
```

```

        x="year",      y="lifeExp",
        color="country",      title="Life
        Expectancy Comparison" )

fig.update_layout(      updatemenus=[      dict(
type="dropdown",      buttons=[
dict(label="All Countries",      method="update",
args=[{"visible": [True, True, True, True]},
{"title": "Life Expectancy Comparison"}]),

dict(label="India",
method="update",      args=[{"visible": [True,
False, False, False]},
{"title": "Life
Expectancy in India"}]),

dict(label="China",
method="update",      args=[{"visible": [False,
True, False, False]},
{"title": "Life
Expectancy in China"}]),

dict(label="USA",
method="update",      args=[{"visible": [False,
False, True, False]},
{"title": "Life
Expectancy in USA"}]),

dict(label="Brazil",
method="update",      args=[{"visible": [False,
False, False, True]},
{"title": "Life Expectancy in Brazil"}])
    ]
)
] )
fig.show()

```

3. Bar Plot with Buttons (Switch Between Variables)

- Plot population (pop) by continent for 2007.
- Add buttons to toggle between population and GDP per capita (gdpPercap).
- Use grouped bars for clarity.
- Title: "Continent Comparison (2007)".
- Label the x-axis as "Continent" and the y-axis as "Value".
- Display the legend at the top-right corner

```
In [5]: df2007 = df[df.year == 2007]
```

```

df_continent = df2007.groupby("continent", as_index=False).agg({
    "pop": "sum",
    "gdpPercap": "mean"
})

fig = px.bar(df_continent, x="continent",
y="pop", color="continent",
title="Continent Comparison (2007)",
labels={"continent": "Continent", "pop": "Value"}
)

fig.update_layout(
    updatemenus=[dict(
type="buttons", direction="right",
showactive=True, buttons=[
dict(label="Population",
method="update", args=[{"y":
[df_continent["pop"]]},
{"yaxis": {"title": "Value"},
"title": "Continent Comparison (2007) -
Population"}]),
dict(label="GDP per Capita",
method="update", args=[{"y":
[df_continent["gdpPercap"]]},
{"yaxis": {"title": "Value"},
"title": "Continent Comparison (2007) - GDP per
Capita"}])
]
)
fig.show()

```

4. Map with Projection Selector

- Plot life expectancy (lifeExp) by country on a world map for 2007.
- Use population (pop) to size the markers.
- Add hover tooltips to show country names.
- Add buttons to switch between map projections (Natural Earth, Mercator, Orthographic).
- Title: "World Map of Life Expectancy (2007)"

In [4]:

```

import plotly.express as px
a = df[df.year == 2007]
fig = px.scatter_geo(

```

```

a, locations="country",
locationmode="country names", size="pop",
color="lifeExp", hover_name="country",
title="World Map of Life Expectancy (2007)" )

fig.update_layout( width=1000,
height=600, updatemenus=[
dict( type="buttons",
direction="right", x=0.5,
y=1.15, showactive=True,
buttons=list([
dict(label="Natural Earth",
method="relayout",
args=[{"geo.projection.type": "natural earth"}]),

dict(label="Mercator",
method="relayout",
args=[{"geo.projection.type": "mercator"}]),

dict(label="Orthographic",
method="relayout",
args=[{"geo.projection.type": "orthographic"}])
])
] )
fig.show()

```

C:\Users\pkg15\AppData\Local\Temp\ipykernel_15172\2019282134.py:5:
DeprecationWarning:

The library used by the `*country names*` ``locationmode`` option is changing in an upcoming version. Country names in existing plots may not work in the new version. To ensure consistent behavior, consider setting ``locationmode`` to `*ISO-3*`.

5. Animated Scatter Plot with Play/Pause Controls

- Create an animated scatter plot of GDP per capita vs life expectancy across years.
- Use population (pop) for point size.
- Color points by continent.
- Add play/pause controls to animate automatically.
- Title: "Global Development Trends (1952–2007)".
- Label axes appropriately.

In [6]:

```

df_1952_2007 = df[(df["year"] >= 1952) & (df["year"] <= 2007)].copy()

fig2 = px.scatter(df_1952_2007,
x="gdpPercap", y="lifeExp",
animation_frame="year",
animation_group="country", size="pop",
color="continent", hover_name="country",
title="Global Development Trends (1952-2007)",
labels={
    "gdpPercap": "GDP per Capita (USD)",
    "lifeExp": "Life Expectancy (Years)",
    "continent": "Continent",
    "pop": "Population",
    "year": "Year",
},
size_max=60,
)

fig2.update_layout(
    updatemenus=[
        {
            "type": "buttons",
            "direction": "left",
            "showactive": False,
            "x": 0.1,
            "xanchor": "right",
            "y": 0,
            "yanchor": "top",
            "pad": {"r": 10, "t": 70},
            "buttons": [
                {
                    "label": "Play",
                    "method": "animate",
                    "args": [
                        None,
                        {
                            "frame": {"duration": 600, "redraw": False},
                            "fromcurrent": True,
                            "transition": {"duration": 300, "easing": "quadratic-
in
            },
        ],
    },
    {
        "label": "Pause",
        "method": "animate",
        "args": [
            [None],
            {
                "frame": {"duration": 0, "redraw": False},
                "mode": "immediate",

```

```

        "transition": {"duration": 0},
    },
],
},
] )
fig2.show()

```

6. 3D Scatter Plot

- Plot GDP per capita (gdpPercap), life expectancy (lifeExp), and population (pop) in 2007.
- Use continent for color coding.
- Size points by population.
- Title: "3D Scatter: GDP, Life Expectancy, Population (2007)".
- Label axes appropriately.

```

In [7]: df_2007 = df[df["year"] == 2007]

fig = px.scatter_3d(df_2007, x="gdpPercap",
y="lifeExp", z="pop", color="continent",
size="pop", hover_name="country", title="3D Scatter:
GDP, Life Expectancy, Population (2007)", labels={
    "gdpPercap": "GDP per Capita (USD)",
    "lifeExp": "Life Expectancy (Years)",
    "pop": "Population",
    "continent": "Continent",
},
size_max=40,
)
fig.show()

```

7. Heatmap

- Compute the correlation matrix for lifeExp, gdpPercap, and pop.
- Plot the matrix as a heatmap.
- Add numeric values inside each cell.
- Title: "Correlation Heatmap of Development Indicators".
- Use a diverging color scale for better contrast.

```
In [15]: corr_matrix = df[["lifeExp", "gdpPercap", "pop"]].corr()

fig3 = px.imshow(    corr_matrix,    text_auto=".2f",
color_continuous_scale="thermal",    zmin=-1,
zmax=1,    title="Correlation Heatmap of Development
Indicators",    labels={"color": "Correlation"},
)

fig3.update_layout(xaxis_title="", yaxis_title="")
fig3.show()
```

8. Pie Chart

- Create a pie chart showing population share (pop) by continent in 2007.
- Add percentage labels to each slice.
- Title: "Population Share by Continent (2007)".
- Use contrasting colors for slices

```
In [19]: pop_by_continent = df_2007.groupby("continent", as_index=False)["pop"].sum()

fig4 = px.pie(    pop_by_continent,
names="continent",    values="pop",
title="Population Share by Continent (2007)",
color="continent",
color_discrete_sequence=px.colors.qualitative.Bold,
)

fig4.update_traces(textinfo="label+percent")
fig4.show()
```

9. Treemap of Population

- Title: "Population Treemap (2007)"
- Size blocks by population.
- Color blocks by continent.
- Hover tooltips should show country names.

In [20]:

```
fig5 = px.treemap(df_2007,
path=["continent", "country"], values="pop",
color="continent", hover_name="country",
title="Population Treemap (2007)",
color_discrete_sequence=px.colors.qualitative.Bold,

)

fig5.show()
```

10. Sunburst Chart

- Title: "Sunburst of Population (2007)".
- Inner ring = continent, outer ring = country.

```
In [21]: fig6 = px.sunburst(df_2007,
path=["continent", "country"], values="pop",
title="Sunburst of Population (2007)",
color="continent",
color_discrete_sequence=px.colors.qualitative.Bold,
)
fig6.show()
```